OPEN IT SUMMER

BOOTCAMP

DAY 8

Web Review: HTML • CSS • JavaScript

Separate files • Linking them • DOM • Events • Fetch API • CORS • Toast Notifications

Duration: ~3 hrs | Build a professional frontend connected to your Day 7 LibraryApi

Roadmap Day 7: API + PostgreSQL → Day 8: Frontend UI → Day 9–10: React replaces plain HTML/JS

Correlates with Day 6 & 7 Student Enrolment schema — same concepts, applied in the browser

Session Overview

01	**HTML — The Skeleton** Semantic tags, attributes, structure, how browser reads HTML
02	**CSS — Styling & Layout** Selectors, box model, Flexbox, Grid, variables, media queries
03	**JavaScript — Behaviour** Variables, functions, scope, arrays, objects
04	**Separate Files & Linking All Three** index.html + style.css + app.js — how they connect
05	**DOM Manipulation In-Depth** querySelector, createElement, classList, dataset, events
06	**Event Handling** addEventListener, form submit, input events, delegation
07	**Fetch API — HTTP from JavaScript** GET, POST, PATCH, DELETE — calling your LibraryApi
08	**Toast Notifications** Building reusable user feedback from scratch
09	**CORS Configuration** Why it blocks, how Program.cs fixes it
10	**Mini Activities + 2-Hour Lab** Day 6/7 Enrolment schema → frontend demo

PART 01 — HTML

The Skeleton of Every Web Page

Tags • Attributes • Semantic Elements • Head vs Body

HTML — How the Browser Reads Your File

HTML is parsed top-to-bottom. Every tag opens a node in the DOM tree.

Full Anatomy of an HTML Document

```
<!DOCTYPE html>
<!-- ↑ Declaration — tells browser: "this is HTML5"
       Must be the VERY FIRST line. Not a tag, just a hint. -->

<html lang="en">
<!-- ↑ Root element — wraps EVERYTHING
       lang="en" tells screen readers the language -->

  <head>
  <!-- ↑ Invisible to users. Contains metadata, links,
         title, character encoding, viewport settings -->
    <meta charset="UTF-8">
    <!-- ↑ Character encoding. UTF-8 supports ALL languages.
           Without this, special chars (é ñ 🌹 日) break. -->

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- ↑ CRITICAL for mobile. Without this:
           • Phone shows desktop layout zoomed out
           • CSS media queries DO NOT WORK
           Include this on every page you ever build. -->

    <title>LibraryApp</title>
    <!-- ↑ Shows in browser tab and search results -->

    <link rel="stylesheet" href="style.css">
    <!-- ↑ Loads your CSS file. href = path to the file.
           Goes in <head> so CSS loads BEFORE the body renders.
           This prevents flash of unstyled content. -->
  </head>

  <body>
  <!-- ↑ Everything visible to the user goes here -->

    <!-- Your page content here -->

    <script src="app.js"></script>
    <!-- ↑ Load JavaScript LAST — at the bottom of <body>
           Why? JS runs immediately when the browser hits it.
           If it's in <head>, the DOM doesn't exist yet
           and querySelector() returns null. -->
  </body>
</html>
```

Key Rules

`lang="en"`

Helps screen readers + SEO

`charset UTF-8`

Required for all languages/emoji

`viewport meta`

Required for mobile + media queries

`<link> in head`

CSS before body = no flash

`<script> last`

DOM exists before JS runs

`One <html>`

One root. Only one <head>, one <body>

Mental model: <head> = instructions for the browser (invisible). <body> = instructions for the user (visible).

Semantic HTML — Use the Right Tag

Semantic = the tag describes its PURPOSE, not just how it looks

Before vs After: Semantic HTML

```
<!-- BEFORE — all divs, no meaning -->
<div class="top">
  <div class="nav">
    <div class="link">Home</div>
  </div>
</div>
<div class="content">
  <div class="article">
    <div class="heading">Latest Books</div>
    <div class="para">Clean Code is a must-read...</div>
  </div>
</div>
<div class="bottom">© 2026</div>

<!-- AFTER — semantic HTML -->
<header>
  <nav>
    <a href="/">Home</a>
  </nav>
</header>
<main>
  <article>
    <h1>Latest Books</h1>
    <p>Clean Code is a must-read...</p>
  </article>
</main>
<footer>© 2026</footer>

<!-- Same visual result. But the second version:
     ✓ Screen readers can navigate it correctly
     ✓ Search engines understand importance of content
     ✓ Other developers understand your intent
     ✓ Browser applies default styling intelligently -->
```

Semantic Tag Reference

`<header>`	Top area — logo, navigation, hero
`<nav>`	A block of navigation links (can appear anywhere)
`<main>`	The PRIMARY content. Only ONE per page.
`<section>`	A thematic grouping with a heading. Like a chapter.
`<article>`	Self-contained content (a blog post, a book card)
`<aside>`	Related but secondary — sidebar, callout boxes
`<footer>`	Bottom of page or section — copyright, links
`<figure>`	Image + optional <figcaption> label below it
`<h1>–<h6>`	Headings. h1 = most important. ONE h1 per page.
`<p>`	Paragraph of text. Block-level.
`<span>`	Inline container. No semantic meaning — for styling.
`<div>`	Generic block container. Use when nothing else fits.

HTML Forms & Common Attributes

Forms are how users send data. Attributes configure element behaviour.

HTML Forms — Every Input Type

```
<!-- FORM — contains input elements
     action: where to send data (we override with JS)
     method: GET or POST (we override with fetch()) -->
<form id="add-book-form">

  <!-- TEXT — single line of text -->
  <input type="text"     name="title"  placeholder="Book title"  required>

  <!-- NUMBER — only allows numeric input -->
  <input type="number"   name="year"   placeholder="Year"
         min="1000" max="2030">

  <!-- EMAIL — validates email format automatically -->
  <input type="email"    name="email"  placeholder="your@email.com">

  <!-- PASSWORD — hides characters -->
  <input type="password" name="pass"   placeholder="Password">

  <!-- SEARCH — shows clear button on some browsers -->
  <input type="search"   id="search"   placeholder="Search books...">

  <!-- DROPDOWN — predefined options -->
  <select name="genre">
    <option value="">-- Select Genre --</option>
    <option value="Programming">Programming</option>
    <option value="Fiction">Fiction</option>
  </select>

  <!-- MULTI-LINE TEXT -->
  <textarea name="description" rows="4"></textarea>

  <!-- CHECKBOX — true/false toggle -->
  <input type="checkbox" name="available" checked>
  <label>Available</label>

  <!-- SUBMIT — triggers the submit event -->
  <button type="submit">Add Book</button>

  <!-- RESET — clears all fields -->
  <button type="reset">Clear</button>

</form>
```

Common HTML Attributes

```
<!-- ID — unique identifier. One per page.
     Used by: getElementById, querySelector, CSS #id -->
<div id="books-grid">

<!-- CLASS — reusable group label. Multiple allowed.
     Used by: querySelectorAll, CSS .class -->
<div class="book-card featured">

<!-- href — where a link goes -->
<a href="https://example.com">External link</a>
<a href="/books">Internal link</a>
<a href="#section-id">Same-page anchor</a>

<!-- src — source file for media -->
<img  src="cover.jpg"   alt="Book cover">
<script src="app.js"></script>

<!-- alt — fallback text for images (accessibility) -->
<img src="cover.jpg" alt="Clean Code book cover">

<!-- data-* — custom data attributes (used by JavaScript) -->
<div class="book-card" data-id="42" data-genre="Fiction">
<!-- Access in JS: element.dataset.id  → "42"
                   element.dataset.genre → "Fiction" -->

<!-- disabled — prevents interaction -->
<button disabled>Loading...</button>

<!-- placeholder — hint text inside input (disappears on focus) -->
<input placeholder="Search books...">

<!-- required — browser validates this field is not empty -->
<input type="text" name="title" required>

<!-- readonly — shows value but user cannot edit -->
<input type="text" value="Robert Martin" readonly>
```

id must be unique on the page. class can be shared. Use id for JS targeting, class for styling multiple elements.

PART 02 — CSS

Styling & Layout

Box Model • Selectors • Flexbox • Grid • Variables • Animations

CSS — Selectors & The Box Model

Selectors target elements. The box model defines how space is calculated.

CSS Selectors

```
/* ELEMENT — all <p> tags */
p { color: #333; }

/* CLASS — elements with class="book-card" */
.book-card { border-radius: 12px; }

/* ID — element with id="books-grid" */
#books-grid { display: grid; }

/* DESCENDANT — <h3> inside .book-card */
.book-card h3 { font-size: 1.1rem; }

/* DIRECT CHILD — only immediate children */
.navbar > ul { list-style: none; }

/* PSEUDO-CLASS — on hover */
.book-card:hover { transform: translateY(-6px); }

/* PSEUDO-CLASS — first/last child */
li:first-child { border-top: none; }
li:last-child  { border-bottom: none; }

/* PSEUDO-CLASS — nth child */
tr:nth-child(even) { background: #F5F5F5; }

/* ATTRIBUTE — input with type="text" */
input[type="text"] { border: 2px solid #E0E0E0; }

/* MULTIPLE — applies to ALL listed */
h1, h2, h3 { font-family: 'Calibri Black'; }
```

The Box Model

```
/* Every element is a rectangular box with 4 layers:
   content → padding → border → margin              */

.book-card {
  /* CONTENT — width/height of the actual content */
  width: 300px;
  height: auto;

  /* PADDING — space INSIDE the border */
  padding: 1.5rem;          /* all 4 sides */
  padding: 1rem 2rem;       /* top/bottom | left/right */
  padding: 1rem 2rem 1.5rem 2rem; /* top right bottom left */

  /* BORDER — the visible line */
  border: 2px solid #CD172D;
  border-top: 4px solid #CD172D; /* only top side */
  border-radius: 12px;          /* rounded corners */

  /* MARGIN — space OUTSIDE the border */
  margin: 1.5rem;           /* pushes away from siblings */
  margin: 0 auto;           /* center horizontally */
}

/* CRITICAL: box-sizing fix */
/* Default: width = content only. Padding ADDS to width.
   Fix:     width = content + padding + border combined */
*, *::before, *::after {
  box-sizing: border-box; /* ALWAYS add this */
}
```

Specificity — Which CSS Rule Wins?

```
/* When two rules target the same element, the MORE SPECIFIC one wins. */
/* Specificity order (lowest → highest): */
/* element (p, div)  →  class (.card)  →  id (#main)  →  inline style  →  !important */

p            { color: black; }   /* specificity: 0,0,1 */
.book-card p { color: navy; }    /* specificity: 0,1,1 ← wins (more specific) */
#hero p      { color: red; }     /* specificity: 1,0,1 ← wins over class */
p            { color: blue !important; } /* ← nuclear option. Avoid. Breaks maintenance. */
```


CSS Custom Properties & Flexbox

Variables let you define your design system once. Flexbox handles 1-dimensional layouts.

CSS Custom Properties (Variables)

```
/* Custom properties are defined in :root — available everywhere */
:root {
  /* COLOURS */
  --red:        #CD172D;
  --navy:       #0E0F30;
  --bg:         #F1F5F9;
  --white:      #FFFFFF;
  --text:       #1A1A2E;
  --muted:      #6B7280;
  --border:     #E0E0E0;

  /* DESIGN TOKENS */
  --radius:     12px;
  --shadow:     0 4px 24px rgba(0,0,0,0.08);
  --shadow-lg:  0 12px 40px rgba(0,0,0,0.15);
  --transition: all 0.25s ease;

  /* SPACING SCALE */
  --space-sm: 0.5rem;
  --space-md: 1rem;
  --space-lg: 1.5rem;
  --space-xl: 2rem;
}

/* USE variables with var() */
.book-card {
  background:    var(--white);
  border-radius: var(--radius);
  box-shadow:    var(--shadow);
  border-top:    4px solid var(--red);
  padding:       var(--space-lg);
  transition:    var(--transition);
}

/* OVERRIDE in a specific scope */
.dark-theme {
  --bg:    #0F172A;
  --white: #1E293B;  /* elements using var(--white) auto-update */
}
```

Flexbox — 1D Layout

```
/* Flexbox: for layouts that go in ONE direction
   (either a row OR a column, not both at once) */

/* CONTAINER properties */
.navbar {
  display: flex;                  /* activate flexbox */
  flex-direction: row;            /* row (default) | column */
  justify-content: space-between;  /* horizontal spacing */
  /* other values: flex-start | flex-end | center |
     space-around | space-evenly  */
  align-items: center;            /* vertical alignment */
  /* other values: flex-start | flex-end | stretch | baseline */
  gap: 1rem;                      /* space between items */
  flex-wrap: wrap;                /* allow wrapping to next line */
}

/* ITEM properties */
.nav-link {
  flex: 1;        /* grow to fill remaining space equally */
  flex-grow: 1;   /* how much to grow (relative) */
  flex-shrink: 0; /* don't shrink below natural size */
  flex-basis: 200px; /* starting size before growing/shrinking */
  align-self: flex-end; /* override container's align-items */
}

/* CENTERING TRICK — the most common use of flexbox */
.centered {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* COLUMN LAYOUT — stack children vertically */
.book-card {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}
/* Push last element to bottom */
.book-card .btn-delete { margin-top: auto; }
```

Flexbox rule: justify-content controls the MAIN axis (flex-direction). align-items controls the CROSS axis. In row: main=horizontal, cross=vertical. In column: main=vertical, cross=horizontal.

CSS Grid & Responsive Design

Grid handles 2-dimensional layouts. Media queries adapt to screen size.

CSS Grid — 2D Layout

```
/* CONTAINER */
.books-grid {
  display: grid;

  /* EXPLICIT columns — 3 equal columns */
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-columns: repeat(3, 1fr);  /* same thing */

  /* AUTO-RESPONSIVE — the magic one-liner */
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  /* auto-fill: create as many columns as fit
     minmax(260px, 1fr): each column min 260px, max 1 equal share
     Result: 1 col on phone, 2 on tablet, 3+ on desktop
     NO MEDIA QUERIES NEEDED */

  gap: 1.5rem;         /* space between cells */
  row-gap: 1rem;       /* space between rows only */
  column-gap: 2rem;    /* space between columns only */
}

/* ITEM spanning multiple columns */
.featured-card {
  grid-column: span 2;  /* takes up 2 columns wide */
  grid-row: span 2;     /* takes up 2 rows tall */
}
```

Media Queries — Mobile First

```
/* MOBILE FIRST: write base styles for phones.
   Add @media rules to ENHANCE for larger screens. */

/* BASE — phones (no @media needed) */
.books-grid {
  grid-template-columns: 1fr;  /* 1 column */
  padding: 1rem;
}

.navbar { padding: 0 1rem; }

/* TABLET — 600px and up */
@media (min-width: 600px) {
  .books-grid {
    grid-template-columns: repeat(2, 1fr);
  }
  .navbar { padding: 0 1.5rem; }
}

/* DESKTOP — 1024px and up */
@media (min-width: 1024px) {
  .books-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

/* WIDE DESKTOP — 1400px+ */
@media (min-width: 1400px) {
  .books-grid {
    grid-template-columns: repeat(4, 1fr);
  }
  .container { max-width: 1400px; }
}

/* DARK MODE — respects OS setting */
@media (prefers-color-scheme: dark) {
  :root { --bg: #0F172A; --white: #1E2938; }
}
```

Transitions & Keyframe Animations

```
.book-card { transition: transform 0.25s ease, box-shadow 0.25s ease; }
.book-card:hover { transform: translateY(-6px); box-shadow: var(--shadow-lg); }
/* ↑ GPU-accelerated. NEVER animate top/left/width/height — those force layout recalc */

@keyframes fadeInUp {                          @keyframes spin {
  from { opacity: 0; transform: translateY(16px); }   to { transform: rotate(360deg); }
  to   { opacity: 1; transform: translateY(0); }    }
}                                              .spinner { animation: spin 0.8s linear infinite; }
.book-card { animation: fadeInUp 0.3s ease forwards; }   /* linear = constant speed */
```

LIVE DEMO — Library App UI

Open index.html →

http://127.0.0.1:5500/index.html

LibraryApp

Books Add Book Search

Book Catalog

Search books...

All Genres

Programming

Clean Code

by Robert Martin

2008

Available

Lend Delete

Fiction

Dune

by Frank Herbert

1965

Available

Lend Delete

← Toast notification

History

Sapiens

by Yuval Harari

2011

Unavailable

Lend Delete

✓ Book added!

← CSS Grid: 3 columns on desktop, 1 on mobile

ACTIVITY — 20 MIN

Build the Library App HTML & CSS

GOAL: Recreate the UI shown in the demo slide — static HTML/CSS only (no JavaScript yet)

SETUP: Create a new folder called LibraryFrontend → open in VS Code
Create two files: index.html and style.css

STEP 1 — index.html: build the semantic skeleton:

- <header> with <nav class="navbar"> containing brand + links
- <main class="container"> with two <section> tags:
  - #books: h1, search input, genre select,
  - #add-book: h2, form with inputs (title, author, genre, year) + submit button
- <footer> with copyright text
- <script src="app.js"></script> at the bottom of body

STEP 2 — style.css: add these styles in order:

- :root { --red: #CD172D; --navy: #0E0F30; } custom properties
- box-sizing reset + body font/color
- .navbar: display:flex + sticky + navy background
- .books-grid: display:grid + repeat(auto-fill, minmax(260px,1fr))
- .book-card: white bg + border-radius:12px + border-top:4px solid var(--red) + hover lift
- .genre-badge pill + .status-available / .status-unavailable colors
- Media queries: 1 col mobile → 2 col 600px → 3 col 1024px

STEP 3: Hardcode 3 book cards directly in HTML with different genres and statuses

ACTIVITY — 20 MIN

Build the Library App HTML & CSS

STEP 4: drop your frontend files into the wwwroot folder and dotnet run serves them automatically.

```
LibraryApi/
 ├── Controllers/
 ├── Models/
 ├── wwwroot/          ← create this folder
 │    ├── index.html   ← create here
 │    ├── style.css    ← create here
 │    └── app.js       ← create here
 ├── Program.cs
 └── LibraryApi.csproj
```

STEP 5: Add this line after builder.Build(), before app.MapControllers():

And Program.cs must look like this:

```
var app = builder.Build();

app.UseStaticFiles();   // ← ADD THIS
app.UseCors("AllowFrontend");
app.MapControllers();
app.Run();
```

```
cd LibraryApi
dotnet run
```


PART 03 — JAVASCRIPT

Behaviour & Interactivity

Variables • Functions • Arrays • Objects • Scope

JavaScript — Core Fundamentals

JS runs in the browser and controls everything interactive on the page

Variables, Types & Functions

```
// ── VARIABLES ──────────────────────────────────────
// let  – can be reassigned
let count = 0;
count = 1;  // fine

// const – cannot be reassigned (use this by default)
const API_BASE = 'http://localhost:5000/api';
// API_BASE = other;  // TypeError if reassigned

// var – old, avoid. Has function scope (not block scope).

// ── TYPES ──────────────────────────────────────────
const fullName = "Ana Reyes";  // string
const year     = 2008;         // number (int AND float are both "number")
const active   = true;         // boolean – true or false (lowercase)
const empty    = null;         // null – intentionally empty
let   missing;                 // undefined – declared but not set

// ── TEMPLATE LITERALS ──────────────────────────────
// Backtick ( ` ) + dollar-sign + curly brace = embed JS
const msg = `Hello ${fullName}, year: ${year}`;
// Output: 'Hello Ana Reyes, year: 2008'
// Any JS expression works inside the curly braces:
const calc = `Sum: ${2 + 2}`;   // "Sum: 4"

// ── FUNCTIONS ──────────────────────────────────────
function greet(name) {          // declaration – hoisted
  return `Hello, ${name}!`;
}
const greet2 = (name) => `Hello, ${name}!`; // arrow function
const square = n => n * n;      // single param, no parens needed
const add    = (a, b) => a + b; // implicit return (no braces)
const fn = (x) => {             // block body – explicit return
  const doubled = x * 2;
  return doubled;
};
const greet3 = (name, title = 'Ms.') => `Hello, ${title} ${name}!`;
// greet3('Ana')         → 'Hello, Ms. Ana!'
// greet3('Ana', 'Dr.')  → 'Hello, Dr. Ana!'
```

Arrays & Objects

```
// ── ARRAYS ─────────────────────────────────────────
const books = ['Clean Code', 'Dune', 'Sapiens'];

// Access
books[0]           // 'Clean Code'
books.length       // 3
books[books.length - 1]  // 'Sapiens' (last item)

// Add / Remove
books.push('Pragmatic Programmer');   // add to end
books.pop();                          // remove from end
books.unshift('New First');           // add to start
books.shift();                        // remove from start
books.splice(1, 1);                   // remove 1 item at index 1

// Iterate
books.forEach((book, index) => {
  console.log(index, book);
});

// Transform
const upper = books.map(b => b.toUpperCase());
const long  = books.filter(b => b.length > 5);
const found = books.find(b => b.includes('Code'));

// ── OBJECTS ────────────────────────────────────────
const book = {
  id:     1,
  title:  'Clean Code',
  author: { fullName: 'Robert Martin' },  // nested object
  available: true,
};

// Access
book.title            // 'Clean Code'
book['title']         // same – bracket notation for dynamic keys
book.author.fullName  // 'Robert Martin' (chained access)

// Optional chaining – safe if author is null/undefined
book.author?.fullName  // 'Robert Martin' or undefined (no crash)

// Destructuring
const { title, author } = book;     // same as: const title = book.title;
const [first, second] = books;      // array destructuring

// Spread operator
const updated = { ...book, available: false }; // copy + override
```

PART 04 — THREE SEPARATE FILES

How HTML, CSS & JavaScript Work Together

index.html + style.css + app.js — one directory, three roles

Separating HTML, CSS & JavaScript

Each file has ONE job. They communicate through the HTML file.

Folder Structure

```
LibraryFrontend/
│
├── index.html      ← Structure
│                     (The skeleton and entry point)
│
├── style.css       ← Presentation
│                     (All colours, fonts, layout)
│
├── app.js          ← Behaviour
│                     (All interactivity, API calls)
│
└── assets/         ← Optional
    ├── logo.png
    └── favicon.ico

/* WHY SEPARATE? */
/* 1. Single Responsibility — each file does one
thing */
/* 2. Maintainability — change CSS without touching
HTML */
/* 3. Caching — browser caches CSS and JS separately
*/
/* 4. Team work — designer edits CSS, dev edits JS */
/* 5. Readability — files stay small and focused */

/* NEVER mix all three in one file on a real project
*/
/* (We started that way to learn, now we separate) */
```

index.html — Links Both Files

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>LibraryApp</title>

  <!-- LINK CSS — in <head> -->
  <!-- rel="stylesheet" tells browser this is CSS -->
  <!-- href is the PATH from index.html to style.css -->
  <!-- Same folder: href="style.css" -->
  <!-- Subfolder:   href="css/style.css" -->
  <!-- Parent:      href="../style.css" -->
  <link rel="stylesheet" href="style.css">

</head>
<body>

  <!-- All your HTML content here -->
  <header>...</header>
  <main>...</main>
  <footer>...</footer>

  <!-- SCRIPT JS — at bottom of <body> -->
  <!-- src is PATH from index.html to app.js -->
  <!-- Bottom of body = DOM exists when JS runs -->
  <!-- type="module" enables ES modules (import/export) -->
  <script src="app.js"></script>

</body>
</html>

/* style.css and app.js DO NOT reference each other.
   They both connect through index.html only. */
```

What Each File Contains

```
/* style.css — ONLY styles */
:root { --red: #CD172D; }

.navbar {
  background: var(--navy);
  display: flex;
}
.book-card {
  border-radius: 12px;
  box-shadow: var(--shadow);
}

/* NO <style> tags in this file */
/* NO JavaScript in this file  */
/* NO HTML in this file        */

/* ———————————————————— */

/* app.js — ONLY behaviour */
const API_BASE = 'http://localhost:5000/api';

async function loadBooks() {
  const res = await fetch(
    `${API_BASE}/books`
  );
  const books = await res.json();
  renderBooks(books);
}

/* NO <script> tags   */
/* NO CSS in this file*/
/* NO HTML in this file*/

loadBooks();
```

Rule: if it looks like something → CSS. If it IS something (structure) → HTML. If it DOES something (interactivity) → JavaScript. Keep the roles separate.

PART 05 — DOM MANIPULATION

Document Object Mode

Selecting • Creating • Modifying • Removing • Reading

What is the DOM?

When the browser loads HTML, it builds a tree of JavaScript objects — one per element. That tree IS the DOM.

HTML → DOM Tree

```
<!-- Your HTML file -->
<body>
  <header>
    <h1>LibraryApp</h1>
  </header>
  <main>
    <section id="books">
      <div class="book-card">
        <h3>Clean Code</h3>
        <p>Robert Martin</p>
      </div>
    </section>
  </main>
</body>

/* The browser parses this and builds a tree:
   Window
   └── document
       └── <html>
           ├── <head>
           └── <body>
               ├── <header>
               │   └── <h1>  text: "LibraryApp"
               └── <main>
                   └── <section id="books">
                       └── <div class="book-card">
                           ├── <h3> text: "Clean Code"
                           └── <p>  text: "Robert Martin"

   Every box is a JavaScript OBJECT.
   JavaScript can read, create, modify, and delete any node. */
```

Accessing the DOM from JavaScript

```
// The DOM is accessed via the global "document" object

// document = the entire HTML page as a JS object
console.log(document.title);    // "LibraryApp"
console.log(document.URL);      // current page URL

// window = the browser window (parent of document)
console.log(window.innerWidth);  // browser width in px
console.log(window.location.href); // current URL

// When you read a value:
const title = document.title;  // reads from DOM

// When you WRITE a value, the page UPDATES LIVE:
document.title = "New Title";   // browser tab changes instantly

// This is what makes web apps "dynamic" — JavaScript
// reads and writes the DOM, and the browser re-renders
// only the affected parts automatically.

// No page reload needed. No server round trip.
// This is different from traditional web pages where
// every interaction reloads the entire HTML from the server.
```

The Three Layers Working Together

```
HTML builds the DOM nodes.   →   CSS styles the DOM nodes.   →   JavaScript manipulates the DOM nodes.
<div class="book-card"            .book-card {                    const card = document.querySelector('.book-card');
     id="card-1">                   background: white;              card.style.borderColor = 'red';      // JS overrides CSS
  <h3>Clean Code</h3>               border: 2px solid navy;         card.classList.add('featured');      // JS adds CSS class
</div>                            }                               card.querySelector('h3').textContent = 'Updated Title'; // JS changes content
```

DOM — Selecting Elements

Every DOM operation starts by selecting the element you want to work with

Selection Methods — All Variants

```
// ── getElementById ──────────────────────────────
// Fastest. Returns ONE element or null.
const grid = document.getElementById('books-grid');
// Returns: HTMLDivElement or null (if id not found)
// Does NOT include the # symbol

// ── querySelector ───────────────────────────────
// Returns FIRST matching element or null.
// Uses CSS selector syntax — most flexible.
const grid    = document.querySelector('#books-grid');  // by id
const first   = document.querySelector('.book-card');   // by class (first match)
const nav     = document.querySelector('nav');          // by tag
const search  = document.querySelector('input[type="search"]'); // attribute
const addBtn  = document.querySelector('button[type="submit"]');
const inside  = document.querySelector('#form .title-input'); // nested

// ── querySelectorAll ────────────────────────────
// Returns NodeList of ALL matches (like an array).
// Returns empty NodeList (not null) if nothing found.
const allCards = document.querySelectorAll('.book-card');
const headings = document.querySelectorAll('h1, h2, h3');

// Iterate a NodeList:
allCards.forEach(card => {
  card.style.opacity = '0.5';
});

// Convert to real array for all array methods:
const cardsArray = Array.from(allCards);
cardsArray.filter(c => c.dataset.genre === 'Fiction');
```

Navigating the DOM Tree

```
// Once you have an element, you can navigate to relatives

const card = document.querySelector('.book-card');

// PARENT — the element that contains this one
card.parentElement          // the .books-grid div
card.closest('.container')  // nearest ancestor matching selector

// CHILDREN — elements directly inside
card.children               // HTMLCollection of child elements
card.firstElementChild      // first child element
card.lastElementChild       // last child element
card.childElementCount      // number of children

// SIBLINGS — same level in the tree
card.nextElementSibling     // next sibling element
card.previousElementSibling // previous sibling element

// QUERYING WITHIN an element (scoped search)
// Searches only inside that element — faster, more precise
const title  = card.querySelector('h3');
const badges = card.querySelectorAll('.badge');
// ↑ Much better than document.querySelector when
//   you know you're looking inside a specific card

// CHECK if something exists before using it
const btn = document.querySelector('#missing-btn');
if (btn) {
  btn.addEventListener('click', handleClick);
}
// Without null check: btn.addEventListener → TypeError: Cannot read
//                     properties of null
```

querySelector vs getElementById: querySelector is more flexible (any CSS selector). getElementById is faster (O(1) lookup). For ids: use getElementById. For anything else: use querySelector.

DOM — Creating & Modifying Elements

Adding new content to the page, reading user input, changing existing content

createElement & appendChild

```
// ── CREATE an element ──────────────────────────
const card = document.createElement('div');
// Creates a <div> in memory. Not yet on the page.

// ── SET properties ─────────────────────────────
card.className      = 'book-card';  // sets class attribute
card.id             = 'card-1';     // sets id attribute
card.dataset.bookId = '42';         // sets data-book-id="42"
card.textContent    = 'Clean Code'; // sets text (safe, no HTML)

// ── innerHTML — set HTML content ───────────────
// ✓ Powerful: can set nested HTML in one shot
// ✗ Danger: if you include unsanitized user input,
//   attackers can inject malicious scripts (XSS)
card.innerHTML = \`
  <span class="genre-badge">Programming</span>
  <h3>\${escapeHtml(book.title)}</h3>
  <p>\${escapeHtml(book.author)}</p>
\`;
// Always escape user-provided data with escapeHtml()

// ── ADD to the page ────────────────────────────
const grid = document.getElementById('books-grid');

grid.appendChild(card);    // add as LAST child
grid.prepend(card);        // add as FIRST child
grid.before(card);         // add BEFORE the grid (sibling)
grid.after(card);          // add AFTER the grid (sibling)

// insertBefore — add before a specific child
const existingCard = grid.firstElementChild;
grid.insertBefore(card, existingCard);  // insert before first

// ── REMOVE from the page ───────────────────────
card.remove();             // remove the element itself
grid.removeChild(card);    // remove a specific child
```

Reading & Modifying Content

```
// ── textContent vs innerHTML ──────────────────
const h3 = document.querySelector('h3');

// READ
h3.textContent   // "Clean Code" — plain text, no HTML tags
h3.innerHTML     // "Clean Code" — includes any HTML tags inside

// WRITE
h3.textContent = '<b>Bold?</b>';
// ↑ Shows literally as "<b>Bold?</b>" — tags are escaped
// ✓ SAFE for user input

h3.innerHTML = '<b>Bold!</b>';
// ↑ Renders as bold text — interprets HTML
// ✗ UNSAFE for user input (XSS risk)

// ── style — direct inline styles ──────────────
card.style.backgroundColor = 'red';    // camelCase!
card.style.display         = 'none';   // hide element
card.style.display         = 'block';  // show element
card.style.opacity         = '0.5';    // semi-transparent
card.style.transform       = 'translateY(-8px)'; // lift

// ── classList — working with CSS classes ──────
card.classList.add('featured');      // add a class
card.classList.remove('featured');   // remove a class
card.classList.toggle('active');     // add if absent, remove if present
card.classList.contains('featured'); // true or false
card.classList.replace('old', 'new'); // swap classes

// ── attributes ─────────────────────────────────
card.getAttribute('data-id')         // get attribute value
card.setAttribute('data-id', '42')   // set attribute
card.removeAttribute('disabled')     // remove attribute
card.hasAttribute('disabled')        // check existence

// ── Form input values ──────────────────────────
const input = document.querySelector('#search');
input.value          // current text in the input
input.value = '';    // clear the input field
input.focus();       // move cursor to this input
```

Never use innerHTML with user-provided data without escaping first. Always use textContent when setting plain text — it is automatically safe.

PART 06 — EVENT HANDLING

Responding to User Actions

addEventListener • Event Object • Form Events • Delegation

Event Handling — Responding to Users

An event is anything that happens in the browser. You "listen" for it and run code in response.

addEventListener — The Standard Way

```
// Syntax: element.addEventListener(event, handler)
// event   = name of the event (string)
// handler = function to run when event fires

const btn = document.querySelector('#load-btn');

// OPTION 1 — inline arrow function
btn.addEventListener('click', () => {
  loadBooks();
});

// OPTION 2 — named function reference (cleaner)
btn.addEventListener('click', loadBooks);

// OPTION 3 — with the event object
btn.addEventListener('click', (event) => {
  console.log(event.target);   // the element clicked
  console.log(event.type);     // "click"
  loadBooks();
});

// REMOVE a listener (must use named function)
btn.removeEventListener('click', loadBooks);

// Common event names:
// Mouse:    click, dblclick, mouseenter, mouseleave, mousemove
// Keyboard: keydown, keyup, keypress
// Form:     submit, input, change, focus, blur, reset
// Window:   load, resize, scroll, DOMContentLoaded
```

The Event Object

```
// Every handler receives an EVENT OBJECT automatically
// It contains information about what happened

document.querySelector('#search')
  .addEventListener('input', (event) => {
    // event.target — the element that fired the event
    const input = event.target;
    console.log(input.value);  // what user typed

    // event.type — what kind of event
    console.log(event.type);   // "input"

    // For keyboard events:
    event.key      // "Enter", "a", "Backspace" etc
    event.code     // "Enter", "KeyA", "Backspace"
    event.ctrlKey  // true if Ctrl was held
    event.shiftKey // true if Shift was held

    // For mouse events:
    event.clientX  // X position on screen
    event.clientY  // Y position on screen
    event.button   // 0=left, 1=middle, 2=right
  });

// Check for Enter key:
input.addEventListener('keydown', (e) => {
  if (e.key === 'Enter') {
    e.preventDefault();  // stop form submit
    doSearch(e.target.value);
  }
});
```

Form Submit — The Critical Pattern

```
/* CRITICAL: Always call e.preventDefault() FIRST in form submit handlers */
/* Without it: the browser submits the form via HTTP → page reloads → all your JS state and DOM content is wiped */

document.querySelector('#add-book-form').addEventListener('submit', async (event) => {
  event.preventDefault();     // ← MUST BE FIRST. Stops the default HTTP form submission.

  // Now safely read the form values:
  const formData = new FormData(event.target);   // event.target = the form element
  const title    = formData.get('title');        // reads input[name="title"]
  const year     = parseInt(formData.get('year')); // parseInt converts "2008" string to number 2008
  const author   = formData.get('author');

  // Or read directly from DOM:
  const title2 = document.querySelector('#input-title').value;

  await createBook({ title, year, author });
  event.target.reset();  // clears all form fields after submission
});
```

Event Delegation — Handling Dynamic Elements

Listen on a parent element to handle events from children that may not exist yet

The Problem with Dynamic Content

```
// PROBLEM: You add event listeners to book cards,
// but new cards are added by JavaScript AFTER the listener is set.

// ✗ THIS DOES NOT WORK for dynamically added cards
const cards = document.querySelectorAll('.book-card');
cards.forEach(card => {
  card.addEventListener('click', handleClick);
});
// querySelectorAll captures elements that exist RIGHT NOW.
// When you add a new card later, it has NO listener attached.

// ✓ SOLUTION: Event Delegation
// Add ONE listener to the PARENT (the grid).
// Events BUBBLE UP from child to parent.
// Check inside the handler which child was clicked.

const grid = document.getElementById('books-grid');

grid.addEventListener('click', (event) => {
  // event.target = the element that was actually clicked
  // It might be the grid, a card, a button, an h3...
  // Use .closest() to find the card ancestor

  const card = event.target.closest('.book-card');
  if (!card) return;  // clicked somewhere outside a card

  const bookId = card.dataset.bookId;

  // Check what was clicked inside the card
  if (event.target.closest('.btn-delete')) {
    deleteBook(bookId);
  } else if (event.target.closest('.btn-toggle')) {
    const isAvailable = card.dataset.available === 'true';
    toggleAvailability(bookId, isAvailable);
  }
});
```

Live Search & Other Input Events

```
// ── LIVE SEARCH — filter as user types ──────────────
let allBooks = [];  // cache full list here

document.querySelector('#search')
  .addEventListener('input', (event) => {
    const term = event.target.value.toLowerCase().trim();

    if (!term) {
      renderBooks(allBooks);  // empty search = show all
      return;
    }

    const filtered = allBooks.filter(book =>
      book.title.toLowerCase().includes(term) ||
      book.genre.toLowerCase().includes(term) ||
      (book.author?.fullName ?? '').toLowerCase().includes(term)
    );

    renderBooks(filtered);  // re-render with matches
  });

// ── GENRE DROPDOWN ──────────────────────────────────
document.querySelector('#genre-filter')
  .addEventListener('change', (event) => {
    const genre = event.target.value;  // selected option value
    const filtered = genre
      ? allBooks.filter(b => b.genre === genre)
      : allBooks;  // empty string = "All Genres"
    renderBooks(filtered);
  });

// ── DEBOUNCE — wait for user to stop typing ─────────
// Without debounce: fires on EVERY keystroke
// With debounce: fires only after 300ms of no typing
let searchTimeout;
searchInput.addEventListener('input', (e) => {
  clearTimeout(searchTimeout);
  searchTimeout = setTimeout(() => {
    fetchBooksFromAPI(e.target.value);  // API call, not local filter
  }, 300);  // 300ms delay
});
```

Event bubbling: click fires on the element clicked, then on its parent, then grandparent, all the way to document. Delegation uses this. event.stopPropagation() stops the bubble if needed.

PART 07 — FETCH API

HTTP Requests from JavaScript

GET • POST • PATCH • DELETE • Error Handling • Async/Await

Fetch API — Making HTTP Requests

fetch() is JavaScript's built-in way to send HTTP requests — same verbs as Postman

How fetch() Works

```
// fetch() is Promise-based and non-blocking.
// When you call fetch(), JavaScript:
// 1. Sends the HTTP request
// 2. Returns a Promise immediately (not the response yet)
// 3. When response arrives, the Promise resolves

// ── GET REQUEST ──────────────────────────────
async function loadBooks() {
  const response = await fetch('http://localhost:5000/api/books');
  // await pauses until HTTP response arrives
  // response = a Response object

  // response.ok     — true if status 200-299
  // response.status — the HTTP status code (200, 404, 500)
  // CRITICAL: fetch() does NOT throw on 404 or 500!
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  // Parse the JSON body
  const books = await response.json();
  // books is now a JS array: [{id:1, title:"Clean Code",...}]
  return books;
}

// response.json()     — parse as JSON (most common)
// response.text()     — parse as plain text
// response.blob()     — parse as binary (images)
```

All CRUD Operations

```
const API = 'http://localhost:5000/api';

// ── GET — read data ──────────────────────────
const res   = await fetch(`${API}/books`);
const books = await res.json();

// GET with query params:
const res2 = await fetch(`${API}/books/search?genre=Fiction`);

// ── POST — create new resource ───────────────
const res3 = await fetch(`${API}/books`, {
  method:  'POST',
  headers: {
    'Content-Type': 'application/json',
    // REQUIRED — tells server the body is JSON
    // Without this: ASP.NET Core gets a null body
  },
  body: JSON.stringify({   // JS object → JSON string
    title:     'Dune',
    author:    'Frank Herbert',
    genre:     'Fiction',
    year:      1965,
    available: true,
  }),
});
// res3.status === 201
// res3.headers.get("Location") has new resource URL

// ── PUT — full replace ───────────────────────
await fetch(`${API}/books/1`, {
  method:  'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: "New Title", ...allFields }),
});

// ── PATCH — partial update ───────────────────
await fetch(`${API}/books/1`, {
  method:  'PATCH',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ available: false }),
});

// ── DELETE — remove ──────────────────────────
await fetch(`${API}/books/1`, { method: 'DELETE' });
// No body needed. Expect 204 No Content.
```

Always JSON.stringify() the body for POST/PUT/PATCH. Always set Content-Type: application/json. Without these, ASP.NET Core receives a null body.

Fetch — Error Handling & Complete Pattern

fetch() only throws on network failure. You must check res.ok for HTTP errors.

Complete Error Handling Pattern

```
// ── WHAT CAN GO WRONG ──────────────────────────────
// 1. Network error  — server unreachable → fetch() THROWS
// 2. HTTP error     — 400/404/500 → fetch() RESOLVES (res.ok = false)
// 3. JSON parse err — body is not JSON → res.json() THROWS

async function loadBooks() {
  showLoading(true);
  try {
    const response = await fetch(`${API_BASE}/books`);

    // MUST check manually — fetch does not throw on 4xx/5xx
    if (!response.ok) {
      const errBody = await response.json().catch(() => null);
      const msg = errBody?.title ?? `HTTP Error ${response.status}`;
      throw new Error(msg);
    }

    const books = await response.json();
    allBooks = books;
    renderBooks(books);
    updateStats(books);
    showSuccess(`${books.length} books loaded`);

  } catch (error) {
    if (error.name === "TypeError") {
      // TypeError = network error (server not running)
      showError('Cannot connect. Is dotnet run active?');
    } else {
      showError(`Error: ${error.message}`);
    }
    console.error("loadBooks failed:", error);

  } finally {
    showLoading(false); // ALWAYS runs — even on error
  }
}
```

POST With Validation Errors + Loading

```
// ── POST WITH VALIDATION ERRORS ────────────────────
async function createBook(data) {
  const res = await fetch(`${API_BASE}/books`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });

  if (res.status === 400) {
    // ASP.NET returns: { errors: { Title: ["required"] } }
    const body = await res.json();
    const msgs = Object.values(body.errors ?? {}).flat();
    showError(msgs.join(" | "));
    return false;
  }
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return true;
}

// ── DISABLE BUTTON DURING REQUEST ──────────────────
async function createBook(data) {
  const btn = document.querySelector('#add-btn');
  btn.disabled    = true;
  btn.textContent = 'Adding...';
  try {
    await submitData(data);
  } finally {
    btn.disabled    = false;
    btn.textContent = 'Add Book';
    // finally ensures button re-enables even if error
  }
}

// ── LOADING SPINNER ────────────────────────────────
function showLoading(show) {
  const spinner = document.getElementById('spinner');
  const grid    = document.getElementById('books-grid');
  if (show) {
    grid.innerHTML = "";        // clear old cards
    spinner.style.display = "flex";
  } else {
    spinner.style.display = "none";
  }
}
```

Always check res.ok before calling res.json(). A 400/404/500 response resolves the fetch() promise — it does NOT throw. Only actual network failures throw.

PART 08 — TOAST NOTIFICATIONS

User Feedback Without Alert()

Building a reusable, animated notification system from scratch

Toast Notifications — Why & What They Are

A "toast" is a small pop-up that slides in, shows a message, then auto-dismisses

Why Not Use alert()?

```
// alert("Book added!") — the browser built-in
alert("Book added!");
// Problems:
// ✗ Blocks ALL JavaScript execution until dismissed
// ✗ User MUST click OK before anything else happens
// ✗ Looks terrible — plain system dialog, ugly
// ✗ Cannot be styled at all
// ✗ Cannot auto-dismiss after a timeout
// ✗ Cannot show multiple notifications at once
// ✗ Annoying on mobile

// console.log() — only shows in DevTools
console.log("Book added!");
// ✗ Users never see this

// SOLUTION: Build a toast notification system
// ✓ Slides in from the corner with animation
// ✓ Colour-coded (green=success, red=error, blue=info)
// ✓ Auto-dismisses after 3 seconds
// ✓ Multiple toasts can appear and stack
// ✓ User can click to dismiss early
// ✓ Fully styled — part of your brand

// Usage once built:
showSuccess('Book added successfully!');
showError('Cannot connect to API.');
showInfo('3 books loaded from database.');
```

Toast Anatomy & CSS

```
.toast {
  position: fixed;   /* stays in place while scrolling */
  bottom: 1.5rem;    /* distance from bottom           */
  right:  1.5rem;    /* distance from right            */
  padding: 0.85rem 1.25rem;
  border-radius: 10px;
  font-weight: 600;
  color: white;
  min-width: 220px; max-width: 360px;
  box-shadow: 0 8px 24px rgba(0,0,0,0.2);
  z-index: 9999; /* on top of everything */

  /* START STATE: off-screen to the right */
  transform: translateX(calc(100% + 2rem));

  /* CSS transition for the slide animation */
  transition: transform 0.35s cubic-bezier(0.34,1.56,0.64,1);
  /* cubic-bezier(0.34,1.56,0.64,1) = bounce effect */
}

/* VISIBLE STATE */
.toast.show { transform: translateX(0); }

/* COLOUR-CODED TYPES */
.toast-success { background: #16A34A; } /* green  */
.toast-error   { background: #DC2626; } /* red    */
.toast-info    { background: #1D4ED8; } /* blue   */
.toast-warning { background: #D97706; } /* amber  */

/* Toast HTML (created by JS — NOT in index.html): */
/* <div class="toast toast-success show">          */
/*   Book added successfully!                      */
/* </div>                                          */
```

The toast element is created entirely by JavaScript — it is not in your HTML file. JavaScript creates it, adds it to the page, animates it, then removes it after 3 seconds.

Toast Notifications — JavaScript Implementation

Step-by-step: create → add to page → animate → auto-dismiss → remove from DOM

showToast() — Full Implementation

```
function showToast(message, type = "success", duration = 3000) {

  // STEP 1: CREATE the div element in memory
  const toast = document.createElement("div");
  toast.className = `toast toast-${type}`;
  // className = "toast toast-success" OR "toast toast-error" etc.
  toast.textContent = message;

  // Optional: add an X button to close early
  const closeBtn = document.createElement("button");
  closeBtn.textContent = "×";
  closeBtn.style.cssText =
    "background:none;border:none;color:white;cursor:pointer;margin-left:0.5rem;font-size:1.1rem;";
  closeBtn.onclick = () => dismissToast(toast);
  toast.appendChild(closeBtn);

  // STEP 2: ADD to the page
  // At this point, CSS puts it off-screen (translateX 100%+)
  document.body.appendChild(toast);

  // STEP 3: TRIGGER the slide-in animation
  // We need a tiny delay (10ms) so the browser has registered
  // the initial off-screen position before we change it.
  // Without this delay: both states render in the same frame
  // and the animation never plays — toast just appears instantly.
  setTimeout(() => toast.classList.add("show"), 10);

  // STEP 4: AUTO-DISMISS after "duration" ms
  const timer = setTimeout(() => dismissToast(toast), duration);

  // STEP 5: Allow clicking the toast to dismiss early
  toast.addEventListener("click", () => {
    clearTimeout(timer);  // cancel the auto-dismiss timer
    dismissToast(toast);
  });
}

function dismissToast(toast) {
  toast.classList.remove("show"); // triggers slide-out CSS transition
  // Wait for the transition (350ms) to finish before removing
  setTimeout(() => toast.remove(), 400);
}
```

Convenience Wrappers & Usage

```
// ── CONVENIENCE WRAPPERS ──────────────────────────
// Call these from anywhere in your code:
const showSuccess = (msg) => showToast(msg, "success");
const showError   = (msg) => showToast(msg, "error");
const showInfo    = (msg) => showToast(msg, "info");
const showWarning = (msg) => showToast(msg, "warning");

// ── USAGE IN YOUR FETCH FUNCTIONS ─────────────────
async function createBook(data) {
  try {
    const res = await fetch(`${API_BASE}/books`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data)
    });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    showSuccess('Book added successfully!');  // ← green toast
    await loadBooks();
  } catch (err) {
    showError('Failed to add book. Is the API running?'); // ← red
  }
}

async function deleteBook(id) {
  if (!confirm("Delete this book?")) return;
  await fetch(`${API_BASE}/books/${id}`, { method: "DELETE" });
  showSuccess('Book deleted.');              // ← green toast
  await loadBooks();
}

// ── WHERE TO PUT THE CODE ─────────────────────────
// showToast() and dismissToast() go in app.js
// .toast CSS goes in style.css
// No changes needed in index.html
// Toasts are created and destroyed by JS only
```

The 10ms delay in setTimeout(() => toast.classList.add("show"), 10) is the most important line. The browser needs one frame to register the starting position before animating to the end position.

PART 09 — CORS

Cross-Origin Resource Sharing

Why the browser blocks, how ASP.NET Core allows it

CORS — Why It Blocks & How to Fix It

CORS is a BROWSER security feature — not a bug. Postman ignores it because Postman is not a browser.

What CORS Is & Why It Exists

```
/* SAME-ORIGIN POLICY (SOP):
   A browser security rule: JavaScript running on
   http://127.0.0.1:5500 (your HTML frontend)
   CANNOT make requests to
   http://localhost:5000 (your API)
   unless the SERVER explicitly allows it.

   An "origin" = scheme + host + port:
   http://127.0.0.1:5500  ← origin of your HTML
   http://localhost:5000   ← different origin (different host+port)

   WHY? Without SOP, any website you visit could
   make API calls to your bank using your cookies.
   e.g., evil.com → fetch("bank.com/transfer?to=hacker")

   CORS = a mechanism for servers to GRANT EXCEPTIONS
   to the Same-Origin Policy.

   The server adds a response header:
   Access-Control-Allow-Origin: http://127.0.0.1:5500
   The browser sees this and ALLOWS the request.

   Without this header → CORS ERROR in browser console. */
```

How Preflight Works

```
/* For "non-simple" requests (POST with JSON body),
   the browser sends an OPTIONS request FIRST (preflight).
   It asks: "Hey server, can I send this?"
   If server says yes → actual request is sent.

   PREFLIGHT REQUEST:
   OPTIONS /api/books HTTP/1.1
   Origin: http://127.0.0.1:5500
   Access-Control-Request-Method: POST
   Access-Control-Request-Headers: Content-Type

   PREFLIGHT RESPONSE (ASP.NET Core handles this):
   HTTP/1.1 204 No Content
   Access-Control-Allow-Origin: http://127.0.0.1:5500
   Access-Control-Allow-Methods: POST
   Access-Control-Allow-Headers: Content-Type

   Then the ACTUAL POST is sent.

   You can SEE this in Chrome DevTools Network tab:
   • First request: OPTIONS (method column)
   • Second request: POST (the real one)
   If OPTIONS returns 404 or has no CORS headers → error. */
```

Program.cs — Complete CORS Setup

```
// STEP 1: Register the CORS policy (before builder.Build())
builder.Services.AddCors(options => {
    options.AddPolicy("AllowFrontend", policy => {
        policy.WithOrigins(
                "http://127.0.0.1:5500",   // VS Code Live Server
                "http://localhost:5500",    // alternate Live Server
                "http://localhost:3000"     // React (Day 9-10)
            )
            .AllowAnyHeader()              // Content-Type, Authorization, etc.
            .AllowAnyMethod();             // GET, POST, PUT, PATCH, DELETE
    });
});

// STEP 2: Apply the middleware (after builder.Build(), BEFORE app.MapControllers())
app.UseCors("AllowFrontend");   // ← ORDER MATTERS. Before UseAuthorization().

// TEST in Chrome DevTools Console:
// fetch('http://localhost:5000/api/books').then(r => r.json()).then(console.log)
// If you see data → CORS works. If you see "blocked by CORS" → check step 2.
```

Postman works without CORS because Postman is not a browser — it has no Same-Origin Policy. Always test the browser too, not just Postman.

ACTIVITY — 25 MIN

Connect Frontend to LibraryApi — Full DOM + Fetch

GOAL: Replace hardcoded cards with real PostgreSQL data. Wire up all interactions.

PRE-CHECK: dotnet run is active in a separate terminal (LibraryApi from Day 7)
CORS is configured in Program.cs for http://127.0.0.1:5500

app.js — write these functions:

1. const API_BASE = 'http://localhost:5000/api';
   let allBooks = [];

2. async function loadBooks() {
   showLoading(true);
   try {
   const res = await fetch(API_BASE + '/books');
   if (!res.ok) throw new Error('HTTP ' + res.status);
   allBooks = await res.json();
   renderBooks(allBooks);
   } catch { showError('Cannot connect to API'); }
   finally { showLoading(false); }
   }

3. function createBookCard(book) — create a div.book-card with innerHTML using template literals
   Include: genre badge, title, author, year, status, Lend button, Delete button

4. function renderBooks(books) — clear the grid, then forEach → appendChild

5. async function deleteBook(id) — confirm() + DELETE fetch + loadBooks()

6. form submit event — e.preventDefault() + POST fetch with JSON body + form.reset() + loadBooks()

7. search input event — filter allBooks array → renderBooks(filtered)

8. loadBooks(); ← at the bottom of app.js — runs on page load

VERIFY: Books load automatically. Add → appears. Delete → disappears.
Restart dotnet run → refresh browser → data still there (PostgreSQL!)

Day 7 Activity — Student Enrolment System

📍 Connects Days 6, 7 & 8

Converting your Day 6 PostgreSQL schema into C# EF Core entities with a console menu

Day 7 Activity Requirements

```
/* FROM DAY 6: You created these PostgreSQL tables:
   students, programs, sections, student_sections, student_grades
   And ran JOIN queries to display enrolment data.

   DAY 7 TASK — Convert to C# EF Core:

   1. MODEL CLASSES (convert each table to a C# class):
      Student    → Models/Student.cs
      Program    → Models/Program.cs  (rename: AcademicProgram)
      Section    → Models/Section.cs
      StudentSection → Models/StudentSection.cs  (junction table)
      StudentGrade   → Models/StudentGrade.cs

   2. DBCONTEXT:
      Create EnrolmentContext.cs with:
      • DbSet for each entity
      • Fluent API for all FK relationships
      • HasData() seed with 4 students, 3 programs, 4 sections

   3. MENU PROGRAM (Program.cs console):
      Console.WriteLine("1. Seed Data");
      Console.WriteLine("2. Show Student Enrolment");
      var choice = Console.ReadLine();

      Option 1: Run seeding (EnsureCreated or migrations)
      Option 2: Query and display student → section → program

   4. SEED FUNCTION:
      Replicates the Day 6 INSERT statements using HasData()
      or by manually calling context.Students.Add() etc.

   5. QUERY FUNCTION:
      Replicates the Day 6 JOIN query:
      SELECT students.*, sections.code, programs.program_name
      FROM students
      JOIN student_sections ON ...
      JOIN sections ON ...
      JOIN programs ON ...
      WHERE students.enrolled = true  */
```

Entity Classes Quick Reference

```
// Models/Student.cs
public class Student {
    public int    Id         { get; set; }
    [Required][MaxLength(50)]
    public string FirstName  { get; set; } = "";
    [Required][MaxLength(50)]
    public string LastName   { get; set; } = "";
    [Range(1,4)]
    public int    Year       { get; set; }
    public string Gender     { get; set; } = "";
    public bool   Enrolled   { get; set; } = true;
    public DateTime CreatedAt { get; set; }
        = DateTime.UtcNow;
    public int? ProgramId    { get; set; }
    public AcademicProgram? Program { get; set; }
    public ICollection<StudentSection>
        StudentSections { get; set; } = new List<StudentSection>();
    public ICollection<StudentGrade>
        Grades { get; set; } = new List<StudentGrade>();
}

// Models/StudentGrade.cs
public class StudentGrade {
    public int    Id        { get; set; }
    public int    StudentId { get; set; }
    public Student Student  { get; set; } = null!;
    [Range(0, 100)]
    public double Grade     { get; set; }
}

// See reviewer for: AcademicProgram,
// Section, StudentSection, EnrolmentContext
```

"Program" is a reserved keyword in C#. Name the class AcademicProgram or SchoolProgram to avoid conflicts with the Program.cs startup file.

2-HOUR HANDS-ON LAB

Connects Day 6 (PostgreSQL schema) → Day 7 (EF Core console app) → Day 8 (Browser frontend)

1

Part 1 — HTML Structure (30 minutes)

Build wwwroot/index.html — the student enrolment dashboard. The page needs: Sticky navbar with the school/system name. A stats bar showing: Total Students, Enrolled, Programs. A search input to filter by name. A program filter dropdownA year filter dropdown (Year 1 / 2 / 3 / 4). A table (not cards this time — tabular data fits a table better) showing each studentEach row: Full Name, Year, Gender, Program, Section, Avg Grade, Status badge

2

Part 2 — CSS Styling (25 minutes)

Build wwwroot/style.css. The main difference from the Library App: this uses a table layout instead of a card grid. Everything else uses the same design system. :root { --red: #CD172D; --navy: #0E0F30; --bg: #F1F5F9; --white: #FFFFFF; --text: #1A1A2E; --muted: #6B7280; --border: #E0E0E0; --radius: 10px; --shadow: 0 4px 24px rgba(0,0,0,0.08);}

3

Part 3 — JavaScript: Fetch + Render + Filter (35 minutes)

Build wwwroot/app.js — fetches from /api/students (relative URL, no CORS needed) and renders the table.

4

Part 4 — Full Stack Verification (30 min)

Open DevTools Network tab. Watch every fetch call. Do the persistence test: Add book → stop API → restart → still there. Open pgAdmin: verify data in PostgreSQL. Test all 4 parts of Day 7 activity menu from browser.

4

Part 4 — Full Stack Verification (30 min)

Test	Expected result
`http://localhost:5000/index.html`	Page loads, spinner shows briefly, table populates
Stats bar	Total = 4, Enrolled = 4, Programs = 3, Avg Grade = average of all grades
Type "ana" in search	Only Ana Reyes row shows
Select "Computer Science" in program filter	Only CS students show
Select "Year 2" in year filter	Only Year 2 students show
Combine search + program filter	Both filters apply at the same time
Clear all filters	All 4 students return
Grade badges	85+ = green, 75–84 = amber, below 75 = red
Resize browser to narrow width	Table switches to stacked card layout
Database verification:	Open pgAdmin → enrollmentdb → run these queries and confirm they match what the browser shows

References & Further Reading

Day 8

[1] **HTML Living Standard**
WHATWG · html.spec.whatwg.org

[2] **CSS Reference — MDN**
developer.mozilla.org · developer.mozilla.org/en-US/docs/Web/CSS/Reference

[3] **DOM API — MDN**
developer.mozilla.org · developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

[4] **Fetch API — MDN**
developer.mozilla.org · developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[5] **CSS Grid — Complete Guide**
CSS-Tricks · css-tricks.com/snippets/css/complete-guide-grid

[6] **CSS Flexbox — Complete Guide**
CSS-Tricks · css-tricks.com/snippets/css/a-guide-to-flexbox

[7] **CORS in ASP.NET Core**
Microsoft Docs · learn.microsoft.com/aspnet/core/security/cors

[8] **addEventListener — MDN**
developer.mozilla.org · developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

[9] **VS Code Live Server Extension**
marketplace.visualstudio.com · Ritwick Dey — right-click index.html → Open with Live Server